



FindinGeo

Prosjektrapport

**Fabian Heflo
Katarzyna Haland
Christian Aron Vea
Harrishan Jegatheeswaran**

Norges miljø- og biovitenskapelige universitet (NMBU)
I samarbeid med Kartverket

Våren 2025

1 Introduksjon

Denne rapporten dokumenterer vårt "Proof of Concept"-studentprosjekt hos Kartverket. Vi er en gruppe studenter ved Norges miljø- og biovitenskapelige universitet (NMBU): Christian, Harrishan, Fabian og Katarzyna. I starten vurderte vi flere ulike prosjekter, men vi valgte å utvikle en spørsmål-svar-chatbot spesifikt for geodata. Målet med chatboten er å forenkle søk i geodata, datasettene til Kartverket og gjøre det enklere for brukere å finne relevant informasjon raskere.

1.1 Valg av navn

Navneprosessen var grundig og tidkrevende, ettersom vi ønsket et navn som både reflekterte chatbotens funksjonalitet, var originalt og lett å huske. Etter mye idémyldring og diskusjoner, endte vi til slutt opp med **"FindinGeo"**.

Navnet er sammensatt av ordene ”find” og ”geo”, noe som understreker chatbotens kjernefunksjon – å finne geografisk informasjon. Vi la også merke til at navnet kan tolkes på flere måter, for eksempel som ”Find in Geo” eller ”Finding Geo”, noe som gir det en dynamisk og fleksibel betydning. Denne lekenheten med ord bidrar til at navnet både er beskrivende og minneverdig.

Ved å velge "FindinGeo" mener vi at å ha funnet et navn som både gjenspeiler prosjektets formål og har et innovativt preg som skaper nysgjerrighet hos brukerne.

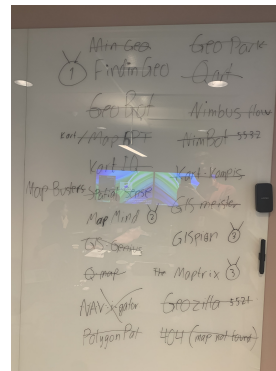


Figure 1: Navneprosess

1.2 Roller

Etter at prosjektet og navnet var fastsatt, fordelte vi de ulike rollene i teamet. Vi ønsket en tydelig ansvarsfordeling for å sikre en strukturert og effektiv arbeidsprosess. De definerte rollene i teamet er "Team Lead", "Tech Lead", "Utvikler" og "Fagspesialist".

Team Lead: Christian tok på seg rollen som Team Lead. Hans ansvar er å holde teamet samlet, sørge for at arbeidet er godt organisert, at backloggen i Jira er oppdatert, og at vi jobber strukturert mot målene våre.

Tech Lead: Katarzyna ble vår Tech Lead. Hun har ansvar for den tekniske oversikten i prosjektet, veileder oss i valg av teknologi og bistår med å løse tekniske utfordringer.

Fagspesialist: Harrishan, som studerer geomatikk, har rollen som fagspesialist. Han sørger for at chatboten håndterer geografisk informasjon på en presis måte og hjelper oss med å tolke og anvende geodata korrekt.

Utvikler: Alle i teamet bidrar som utviklere, der vi sammen arbeider med kodeutvikling, testing og implementering av funksjonalitet.

1.3 MoSCoW Prioritering

For å strukturere og prioritere utviklingen av chatboten har vi brukt MoSCoW-metoden. Prioriteringene er delt inn i fire kategorier:

Kategori	Beskrivelse
Must Have	- Databaser (kartdata) - Lokal/vektor embedding - Språkmodel - Test og trenings data (geodata) - Spesifikk test og trenings data til datasettet vårt - Databeskyttelse
Should Have	- Frontend - Norsk og engelsk - Tilbakemelding fra chatboten ved "feil" input - Returnere query / sikkerhetsprosent på svar - Raske svar
Could Have	- Vise informasjon i kart - Chat-minne - Se på data fra bestemt tid - Generalisere over flere kartdata/databaser
Won't Have	- Tale - Feil/urelevant svar - Real time oppdatering - App

Table 1: MoSCoW-prioritering av funksjoner i FindinGeo

1.4 Backlog i Jira

Med MoSCoW-prioriteringen på plass kunne vi starte arbeidet med backloggen. Vi opprettet prosjektet i Jira og strukturerte arbeidet ved å definere flere epics, som representerte hovedfunksjonaliteter i chatboten. Deretter brøt vi disse ned i mindre, håndterbare tasks, slik at vi fikk en mer oversiktlig og iterativ utviklingsprosess.

Underveis i prosjektet ble backloggen stadig oppdatert. Da vi møttes kun en dag i uken, var dette en trygg havn å starte dagen ved. Slik fikk vi en kort gjennomgang av hva vi jobbet med sist, hva vi skulle gjøre i dag, og et bilde av hva de andre i teamet jobbet med.

1.5 Flow chart

Her er en enkelt visualisering av hvordan chatbotens skjelett ser ut.

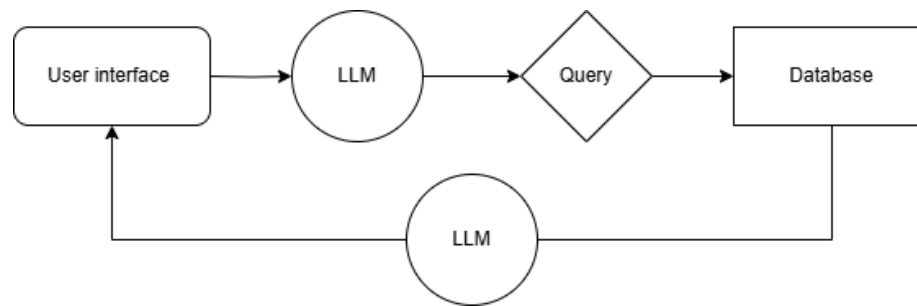


Figure 2: Enkelt flytdiagram for FindinGeo

2 Hva vi har gjort

Å lage en chatbot krever sitt arbeid. Her er en oversikt over hvordan vi har gått frem og hva vi har gjort i prosessen.

2.1 Databaser og datasett

Valg av datasett landet på "forenklet-elveg linjer" og "fotroute", begrenset til Ås kommune. Tanken bak valget gikk ut på at det finnes mange relevante spørsmål forbundet med veier. Veier gir også et bredt utvalg av mulige spørsmål. Disse to datasettene er av betydelig størrelse, særlig "forenklet-elveg linjer", som er alt for stort til det vi trenger i et "prof of concept". Derfor så vi oss nødt til å begrense de, og Ås kommune ble et naturlig avgrensning.

Kartverket satt opp for oss en lokal database, som vi kunne håndtere i pgAdmin 4. Her lagde vi to databaser: dev og prod. Dev er ment til å prøve ut ulike ting - teste hva som fungerer og hva som ikke fungerer. Når det er ferdigutviklet, legges det inn i prod. Prod er det ferdige produktet.

Størrelse på databasen viste seg nødvendig å utvide allerede første dagen, da serveren tok kvelden når vi ved et uhell la inn hele "forenklet-elveg linjer". Kartverket utvida minne for oss, fra 5Gb til 10Gb, og problemene har uteblitt siden.

Vi fikk også tilgang til Kartverkets Fellesbase. Det var her vi endte opp med å hente tabellene fra. Den originale planen var å laste de ned fra Geonorge.no, men dette viste seg vanskelig, da det ikke fantes noen god metode å legge tabellene inn i vår lokale database på. Eneste måte var manuelt, som var en svært tidkrevende prosess. Løsingen ble å lage et script som bruker PostGIS til å lese tabellene fra Fellesbasen, for så å importere de til lokal-databasen. PostGIS fungerer kun på tabeller som har en kolonne for geometri. Tabeller uten geometri blir behandlet av en standard pandas read-sql function.

Det var noen linkede tabeller til datasettene våre. Til "fotroute" var disse tilgjengelige i Fellesbasen, som vi hentet ut på samme måte som hovedtabellene. Derimot lå det ikke inne for "forenklet-elveg linjer", og vi ble nødt til å lage disse selv. Her leste vi av dokumentasjonen, som vi fant på Geonorge, la de inn i et excel-dokument og importerte den direkte gjennom pgAdmin.

Videre i arbeidsprosessen viste det seg at gruppa hadde tid til å utvide fokusrådet til alle kommuner i Akershus fylke. Det innebærer 21 kommuner, der Ås er en av dem. Gruppa fikk lagt til datasett for både fotroute og forenklet elveg. For å bruke data fra fylken ble det skrevet script i python som velger utvalgte kategorier fra tabellen i fellesbasen, og velger data som tilhører Akershus. Kolonnene fra datasettet er de samme som er brukt i de andre datasettene, i tillegg til kolonne for kommune.

I vårt prosjekt har vi følgende linkede tabeller:

- fotrute-aas-malemetode (fra Fellesbasen)
- fotrute-aas-rutefolger (fra Fellesbasen)
- forenklet-elveg-aas-vegfase (lagd selv)
- forenklet-elveg-aas-vegkategori (lagd selv)
- forenklet-elveg-linjer-akershus (fra Fellesbasen)
- fotrute-akershus (fra Fellesbasen)

2.2 Spøringer til våre data

Underveis i ferdigstillingen av databasen hadde vi en mann arbeidende med å lage spøringer til "fotrute" og "forenklet-elveg linjer". Disse spørringene gis til SQL-agenten som eksempler på hvordan den skal formulere sine svar. Hver spørring består av to deler:

1. **Input** - spørsmål på norsk/engelsk, som én bruker ønsker svaret på. F.eks.: *Hvor mange fotruter er det i Ås?*
2. **Query** - spørringen som hører til spørsmålet. F.eks.: *SELECT COUNT(objid) FROM fotrute-aas;*

Modellen takler best spørsmål den har sett før eller som ligner på noe den har sett før. Vi utformet derfor 29 spøringer fordelt på de to datasettene. Disse spørsmålene omhandlet: antall, lengde, posisjon, dato, konkrete tabeller og spørsmål som krever beregning. Vår testing viste at dette fungerte godt.

Vi brukte aktivt QGIS til å dobbeltsjekke at spørringen vi skrev var gyldige. Dette fant vi svært nyttig og forsikret oss om god inndata til modellen.

2.3 Testing av LLMs

Valg av Large Language Model (LLM) er avgjørende for hvorvidt systemet kan gi brukeren korrekte og presise svar basert på datagrunnlaget. Under utviklingen har vi testet flere åpne og gratis tilgjengelige modeller. Det viktigste kriteriet har vært om modellen klarer å forstå spørsmålet og generere korrekt SQL-spørring og svar.

Flere av de testede modellene har hatt utfordringer med enten syntaks, språklig forståelse eller begrenset kontekstkapasitet. For å finne frem til modeller som fungerer tilfredsstillende, har vi jobbet iterativt med testing, evaluering og prompt-justeringer. I tillegg har vi vært nødt til å ta hensyn til token-begrensninger og ytelse, ettersom mange gratisversjoner av modeller har strenge grenser på hvor mye tekst som kan behandles i én forespørsel.

Valg av språkmodell er samtidig en dynamisk prosess. Modellene oppdateres og forbedres kontinuerlig, samtidig som nye alternativer blir tilgjengelige. Det

er derfor viktig å se på modellvalg som en midlertidig beslutning som kan og bør revurderes i takt med utviklingen i feltet. I vårt arbeid har vi derfor hatt et bevisst forhold til modularitet, slik at det enkelt skal kunne byttes mellom modeller ved behov.

2.4 SQL-agent

Vi har ikke testet ut ulike agenttyper, men valgte å benytte en *zero-shot ReAct*-agent som en del av LangChain-biblioteket. Denne typen agent er utviklet for å kombinere resonnering og verktøybruk i flere steg, noe som gjør den godt egnet til å oversette naturlige spørsmål til SQL-spørringer og hente ut relevante svar fra databasen.

Valget ble tatt basert på agentens evne til å utføre komplekse oppgaver uten forhåndstrening på spesifikke eksempler, noe som samsvarer med vår målsetting om å gjøre geografiske data tilgjengelig for brukere uten teknisk kompetanse. Ettersom prosjektet er i en tidlig fase, har vi fokusert på å etablere en fungerende struktur og vurderer å teste og sammenligne andre agenttyper i senere utviklingsfaser.

Et alternativ til agent kunne vært å benytte en enklere `SQLDatabaseChain`, som direkte oversetter input til en SQL-spørring og returnerer resultatet. Fordelen med å bruke en agent er imidlertid at den muliggjør mer avansert kontrollflyt. Den kan for eksempel dobbeltsjekke spørringer før de kjøres, hente ut tabell- eller skjema-informasjon ved behov, og tilpasse svar basert på flere steg med resonnement. Dette gjør løsningen mer robust, spesielt når brukerspørsmålene varierer i kompleksitet.

2.5 Interface

Bruker grensesnittet er en sentral del av brukeropplevelsen, og vi har lagt vekt på å utvikle et intuitivt og visuelt tiltalende design. Vi ønsker å gjøre det tydelig for brukeren hvordan systemet resonnerer, ved å vise frem agentens stegvise tenkning og beslutningsprosess i terminalen eller i chatbot-visningen.

Et viktig mål har vært å presentere både SQL-spørringen og det genererte svaret på en oversiktlig og forståelig måte. Dette bidrar til transparens, og gjør det enklere for brukeren å forstå hvordan svaret er fremkommet.

Videre er det ønskelig at systemet også skal kunne visualisere informasjonen geografisk i et kart. Dette vil gjøre resultatene mer tilgjengelige og relevante, særlig når det gjelder spørsmål knyttet til geodata.

For å implementere brukergrensesnittet har vi benyttet Streamlit. Dette rammeverket gjør det enkelt å bygge interaktive webapplikasjoner i Python, og har vært godt egnet til å koble sammen agentens bakgrunnsprosess med et responsivt og brukervennlig frontend.

2.6 Prompting-strategi

Vi har benyttet en *few-shot prompting*-strategi, der vi gir modellen flere eksempler på spørsmål og tilhørende SQL-spørringer. Dette gir modellen et grunnlag for å forstå både konteksten og strukturen på ønskede svar, og fungerer som en form for kontekstuell læring i sanntid.

I tillegg til eksemplene har vi strukturert prompten med en tydelig beskrivelse av agentens oppgaver og ansvar. Vi spesifiserer hvilke steg modellen skal følge i resonnementet, og gir en liste med retningslinjer som begrenser og styrer hvordan modellen skal svare – for eksempel ved å unngå DML-operasjoner og ikke oppfinne ny informasjon.

Siden vi benytter en ReAct-agent, har vi også inkludert en oversikt over tilgjengelige verktøy i selve prompten. Dette gjør at modellen kan velge riktige handlinger basert på problemet som skal løses. Resonneringsprosessen er definert eksplisitt i prompten, der modellen skal følge et mønster med *Thought*, *Action*, *Observation* og avslutte med *Final Answer*.

Denne strukturerte tilnærmingen har vist seg å være avgjørende for at modellen skal kunne generere presise og trygge SQL-spørringer basert på naturlig språk.

3 Videreutvikling av FindinGeo

FindinGeo har stort potensialet til videreutvikling. Her er vår visjonen om hva vi ser for oss FindinGeo kan bli til.

3.1 Databaser

Målet er å kunne søke i alle databasene til Kartverket, slik at enhver ansatt kan få nytte av FindinGeo, uansett divisjon eller avdeling. FindinGeo skal også henge med på fortløpende oppdateringer i databasene.

3.2 Kjøretid

I dag tar det lang tid for modellen å gi et svar, i hvert fall lengre enn ønskelig. Så det vil være essensielt å optimalisere prosesser og få ned kjøretiden.

3.3 Interface

Her ser vi for oss en integrert løsning, f.eks.: en app, som ligger lett tilgjengelig på en brukers enhet.

Brukeren skal kunne føre en samtale, altså stille oppfølgingsspørsmål eller kommentere svar, med chat-boten. Med andre ord, gi FindinGeo chat-minne.

En hendig ting vi har sett for oss er om svarene kan vises direkte i et kart. Om du spør om hva den lengste elva er i Telemark, så får du ut svaret samt en mulighet for å se den i kartet.

3.4 Språk

Per i dag kan man få ut svar og spørre på norsk og engelsk. Chatboten gir noe inkonsekvent i hva språk den svarer på. Her hadde det vært fint med en funksjon som bestemte hvilket språk man kommuniserer på. Kanskje i form av en knapp, hvor bruker selv velger språket, eller kanskje helst at man får svar på samme språk som man stiller spørsmål på.

Utvidelse av språk hadde også vært interessant å få med - særlig nynorsk i første omgang.

Kjøre

3.5 Bedre nøyaktighetsrapport

FindinGeo produserer i dag en noe kryptisk tekst, som bruker kan lese, i prosessen av å komme frem til et svar. Vi ser forbedringspotensialet her, ved at teksten blir ryddigere og lettlest. De fleste brukere vil trolig ikke være så interessert i å se hva som skjer i bakgrunnen. Så muligens kan man ha dette skjult, men med mulighet for å vise det for den interesserte bruker. Uansett burde man ha et symbol/animasjon som viser at modellen jobber og har progresjon.

3.6 Tale

Det å kunne snakke til FindinGeo og få et svar i samme format ser vi for oss vil være kjekt for noen. Det vil være raskere enn å skrive inn lange spørsmål, og folk som er ute/i felt uten tilgang på et tastatur eller av andre årsaker ikke har enkelt for å skrive, vil kunne operere effektivt med en talefunksjon.

3.7 GeoGPT-integrasjon

GeoGPT er et prosjekt som har som mål å forenkle prosessen med å finne frem til riktig geografisk datasett basert på brukerens behov. Dersom FindinGeo kobles opp mot hele Kartverkets database, kan det oppstå utfordringer knyttet til skala og relevans. Det finnes et stort antall datasett, og uten en tydelig spesifisering av hvilke data som er aktuelle, kan det bli vanskelig for språkmodellen å navigere effektivt og hente ut korrekt informasjon.

En mulig løsning kan være å integrere FindinGeo med GeoGPT, eller etablere et samarbeid mellom de to systemene. GeoGPT kan da brukes som et forfilter, som lokaliserer og foreslår det mest relevante datasettet basert på brukerens forespørsel. FindinGeo kan deretter gå mer i dybden i dette datasettet, og hente ut presise og tilpassede svar ved hjelp av SQL-agenten.

Strukturen ligner på det som i litteraturen omtales som en *Retrieval-Augmented Generation* (RAG)-modell. Her fungerer GeoGPT som et retrieverbasert system som lokaliserer relevante datasett, mens FindinGeo fungerer som det generative systemet som analyserer og henter ut spesifikk informasjon fra disse datasettene.

En slik arbeidsdeling vil kunne gi en mer robust og skalerbar løsning, der brukerens behov dekkes både på overordnet og detaljert nivå.

Under er et flytdiagram av hvordan denne sammenslåingen kan se ut.

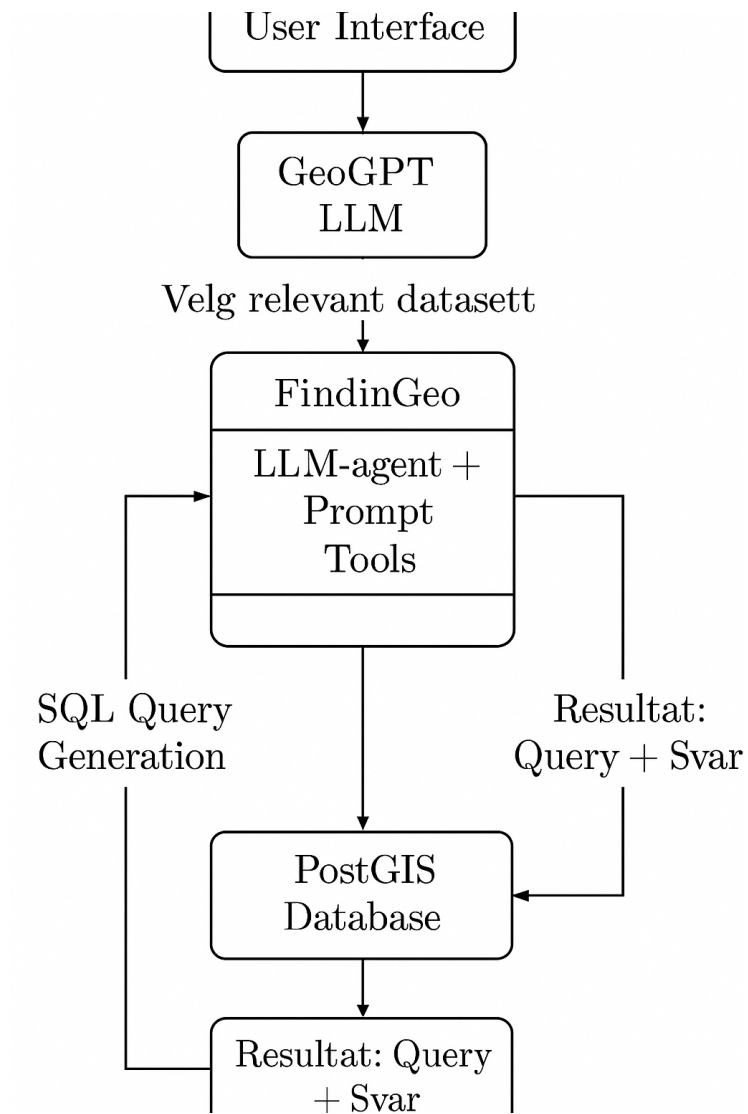


Figure 3: Flytdiagram for samspillet mellom GeoGPT og FindinGeo

4 Tilbakemelding til Kartverket

Vi har noen tilbakemeldinger vi håper Kartverket tar til etterretning, særlig om de skulle samarbeide med NMBU-studenter i fremtiden.

4.1 Utstyr

Teknisk utstyr bør være på plass før oppstart av prosjektet. Det bør også bli gitt anviste plasser studentene kan oppbevare disse når det ikke er i bruk.

4.2 Arbeidsdag

Det at vi har fått komme en fast dag i uken til Kartverkets lokaler i Oslo har vært helt ideelt. Det er et overkommelig tidsbruk for en student til å sette av til et slikt prosjekt, samtidig som én får god kontinuitet i arbeidet.

4.3 Tog

Studenter burde få tilbud om dekning av kollektivtransport fra Ås til Skøyen.